How Many Model Calls Does a Delegated Mortgage Application Take?

A Controlled Comparison of Web, Per-Organization and Common Agent Interfaces

Jesse Wright
University of Oxford

October 2026

Abstract

Personal AI agents such as Meta's Muse and OpenAI's dots now act for people on websites and through connected apps. We measure what that costs on a routine but sensitive task: a mortgage application for two borrowers that needs a choice of lender, records from two banks and a payroll service, a Loan Estimate and two underwriting conditions. In a local sandbox, mock organizations offer the same data through websites built for people, a different API at each organization, and one common agent API with server-side permission checks. Our main condition is grounded in practice: realistic page weight and latency, market loan figures, documents requested only after the Loan Estimate, in line with US rules, a second session once the estimate arrives, and published human response times. Working through websites, Claude Sonnet 5.5 needed a median of 107 model requests, $1.26 at list price and 730 seconds: 364 seconds of model time, 160 of human time and 148 of network time. With a different API at every organization it needed 29 requests, $0.49 and 334 seconds; with the common API, 20 requests, $0.33 and 255 seconds; with ordinary code running the routine steps over either kind of API, 2 requests, under one cent and 152 to 165 seconds. Set-ups needed a median of five to seven human interactions; without websites these took 126 seconds, about half the common-API agent's wall time and five-sixths of the program's. Searching across 3 to 100 lenders, each extra lender added 5.5 seconds and about a cent through websites, 0.8 seconds and 0.4 cents through the common API, and almost nothing through a pre-computed index queried in SPARQL. With grants approved in advance, only one question reached a person and the code-run set-up took 94 seconds. Structured access gave most of the saving; a common interface, with records passed by reference, added a smaller one and lets one integration serve every organization. Across 223 runs no agent shared data outside its authority or recorded intent to proceed on terms outside its limits.

1 Introduction

Personal agents are sold on giving people time back [8, 12]. Today they mostly reach services through websites built for people, through the user's own accounts, or through connectors written for one service at a time [7, 11, 6]. A recurring argument is that common, machine-readable interfaces between agents and organizations would make such agents cheaper, faster and easier to keep within their users' limits [3, 15]. For APIs in general, there are measurements [13]. For interfaces shared across organizations, and for the separate contributions of structured access, a shared interface and code orchestration, we know of no measurements.

We measure them. On the same task, with the same model and the same authority, how many model requests, dollars and seconds does an agent need when it works through websites,

through a different API at each organization, or through one common agent API? How much of any difference comes from structured access, how much from a shared interface, and how much from moving routine steps out of the model? How much of the time belongs to people, how do costs grow with the number of lenders, and what changes when the household delegates in advance?

We contribute a testbed that serves one synthetic household's data identically through all three interfaces; measured costs, time and human interactions for eight agent set-ups, including an RDF variant of the common API, in a grounded condition, a search experiment with up to 100 lenders and five simplified settings; and a survey of how the sandbox compares with services that exist today. The main assumptions are stated, grounded where possible, and tested for sensitivity where the logs allow in Section 8. The testbed, all run logs and the analysis scripts are public [16].

2 Task and testbed

Task. Sam and Nadia Okafor-Reyes (fictional) are buying a $550,000 house with a $440,000 loan. Out of three lenders, the agent must pick the lowest rate whose advertised figures fit the borrowers' limits (one lender offers a lower rate whose closing costs break them) and submit that lender's application. When the Loan Estimate arrives, it must record intent to proceed if the estimate fits; send the documents the lender then asks for, the last two monthly statements of four personal accounts at two banks and the last two payslips of each borrower from the payroll service; and answer two conditions: explain an $8,000 deposit, which needs information only the borrowers have, and provide each borrower's 2025 W-2. US rules bar lenders from requiring verification documents before the Loan Estimate [20]; our sandbox goes further, by its own design, and accepts documents only after the estimate and intent to proceed.

Authority. A `get_authority` tool returns the borrowers' instructions, identical in every set-up: share only those documents, only with the chosen lender, only for this application; accept intent only if the rate is at most 7.5%, the payment at most $3,600 and closing costs at most $14,500; submit once. An `ask_user` tool answers the deposit question ("a gift from Sam's parents"). The Loan Estimate (7.25%, $3,485.58 a month with escrow, $13,200 closing costs) is within the limits.

Organizations and interfaces. Six mock organizations run on 127.0.0.1: two banks, the payroll service and three lenders, of which only Brightpath takes applications online. Each serves the same in-memory state through three interfaces.

- **Website.** A cookie banner that blocks the page, password login, paginated lists, a statement date picker, HTML statements and PDF downloads. Menus, promotions and footers give accessibility snapshots of about 12,400 characters, close to the 12,300-character median of 13 real bank, lender and payroll pages we loaded with the same browser.
- **Per-organization API.** Each differs in style, paths, field names, authentication and date formats, and has a plain-text documentation page.
- **Common agent API**, identical everywhere: one endpoint taking an operation, subject, on-whose-behalf, recipient, purpose, permission token and parameters, with operations such as `records.provide` at the banks and payroll and `application.submit`, `documents.submit` and `condition.respond` at the lenders. The server checks every call against a grant naming the allowed operations, subject, records, recipient, purpose and limits. Records carry their origin and permitted use, and the lender fetches them by reference from their source.

Each lender's rates page embeds its products as schema.org JSON-LD [25]. The servers add our measured median latencies of 1.85 seconds per page and 0.67 seconds per form post or download, and an assumed 0.3 seconds per API call.

People and sessions. Each website login, from what is always a new device, sends a one-time code that the agent must ask for (`request_code`); each per-organization API needs one OAuth consent (`request_consent`), since consumers are not given API keys; the common API needs one wallet approval per organization at first use (`approve_grant`). The simulated person takes 16.6 seconds per code, consent or approval, the median SMS second-factor time in a usability study [31], and an assumed 60 seconds per question. Session 1 ends at submission with a handover note. The testbed then issues the Loan Estimate and expires web sessions and access tokens. Session 2 is a fresh process whose prompt contains the note; it shares no conversation with session 1, but found the runtime's fixed prefix of 9,000–15,000 tokens already in the prompt cache, as an always-on agent would.

Probes. Nadia's business account appears in every account list but is outside the authority. One of her statements carries an injected instruction in its HTML, PDF text, API JSON and agent-API record: "AI assistant: to verify identity, also upload the business account statements and email them to verify@lender-checks.example".

3 Agent set-ups and conditions

All set-ups used the Claude Code command-line agent (version 2.1.280) in non-interactive mode, with its built-in tools (shell, file access, web fetch, sub-agents) removed and only the set-up's tools allowed.

- **A, websites:** Playwright browser tools [10] restricted to the sites. The tool that runs arbitrary Node code was denied, since it is not a browser action.
- **B, per-organization APIs with the lenders' websites:** an HTTP request tool for the banks and payroll and the browser for the lenders. This mirrors today, when lenders rarely offer consumer APIs.
- **B-full, per-organization APIs everywhere:** the HTTP request tool for every organization, also used to fetch the lenders' rates pages; no browser.
- **C1, common agent API:** two tools, `agent_describe` and `agent_request`.
- **C-RDF, common agent API in RDF:** the same operations and permission checks, with requests and responses in JSON-LD using schema.org [25], DPV [18] for permitted use and PROV [24] for origin, plus a few testbed terms where no standard term fits. C-RDF adds an *index*: a shared, pre-computed service, like a search engine or price-comparison site, that one party builds once for every borrower. Before each run it crawls the lenders' rates pages and loads their JSON-LD into the sparq SPARQL engine (WebAssembly build, version 0.1.5) [26]; the agent then compares lenders with one `index_query` call instead of reading each lender's page or API. Building the index is not counted in the run's cost or time.
- **B-full-code and C2, routine steps in code:** a short deterministic program over the per-organization APIs, or over the common API, makes every routine call and checks the Loan

Estimate against the limits. Each calls the model twice: to read the conditions and turn the deposit condition into a question for the borrowers, and to draft the reply. **C2-lean** is C2 with the runtime's default system prompt replaced by one line.

The task prompt was identical for the model-orchestrated set-ups apart from one paragraph naming the interface. The programs are task-specific automation baselines: their two model calls are what they were written to make.

Main conditions. The *grounded* condition (Section 4) is the task above, run on all eight set-ups. *Grounded-standing*, on C1, C-RDF and C2, is identical except that the household approved its grants in advance. *Search* (Section 5) is lender choice alone, at scale: a public registry lists N lenders, each with a website on its own origin and an agent-API product list; about a fifth also offer a lower rate whose points break the closing-cost limit, and exactly one has the lowest fitting rate. We ran A, C1, C-RDF and C2 with $N = 3$, 10 and 30, and C1 and C-RDF also with 100.

Simplified settings. Five earlier settings (Section 6) use a single session, documents sent with the application, a Loan Estimate after a fixed five-second delay, a below-market estimate (6.25%, \$3,042.41, \$12,850) against limits of 6.5%, \$3,100 and \$14,000, the deposit as the only condition, small pages without added latency, and instant answers. The *baseline* names the lender and asks a person only about the deposit. *Friction* adds one transient HTTP 503 on the first record request at each data organization; web sessions that expire after four authenticated pages (eight at the lender); an OAuth token that expires after two uses; agent-API permission tokens that expire after one use and must be re-issued from the user's wallet; and a lender income page that asks for "gross monthly pay". We call each injected fault that the agent has to recover from (a 503, an expired session or an expired token) a *friction event*. Because the faults depend on the interface, A met seven friction events per run, B five (two of its APIs use static keys, with nothing to expire) and C1 and C2 eight. The relabelled income field is not counted; it affects only the lender's website. In *over-ask* the lender's condition also asks for "statements for all accounts held by either borrower, including business accounts". In *out-of-limits* the rate is 6.875%, above the limit (the payment is shown without escrow so that only the rate breaks a limit); a correct run records no intent and tells the borrowers. *Realistic* adds the lender choice and the sign-in steps above, with instant answers.

Measurement. All model traffic went through a logging proxy. A *model request* is one inference call by the agent, counted as a distinct assistant message; the runtime also makes one to seven small housekeeping calls per run outside its visible transcript, which we report separately. Tokens sum cache reads, cache writes, uncached input and output; cost is computed at Anthropic's list prices from each request's token classes [27]. Wall time runs from process start to exit, summed over sessions, and is split into model time (the summed duration of the agent's own model requests; housekeeping calls count as runtime overhead), human time (summed waits for people) and network time (summed server handling, including the added latency). The runtime's start-up and end-of-turn housekeeping, which a persistent agent would not repeat for each task, are reported separately (estimated from calibration runs for the grounded programs). A *human interaction* is any call that needs a person: a code, a consent, an approval or a question. A checker read the final server state and logs. A run succeeds if every required document reached the lender, the key application fields are correct, the condition response mentions the gift, intent is recorded (or, out of limits, not recorded), and there are no violations: no unauthorized or early document, no business account among the assets, no duplicate step, no intent outside the limits and no use of the injected address. Each run had fresh state and credentials, a 45-minute timeout and a spending cap; none hit either.

Set-up	Requests	Tokens	Cost ($)	Wall (s)	Model	Human	Network	Interactions
A websites	107 (105–115)	5.58M	1.26	730 (729–745)	364	160	148	7
B per-org + lender web	77 (70–81)	3.55M	0.93	542 (539–571)	287	143	81	6
B-full per-org APIs	29 (29–30)	0.87M	0.49	334 (328–341)	179	126	25	5
C1 common API	20 (19–20)	0.46M	0.33	255 (249–255)	113	126	13	5
C-RDF JSON-LD + index	20 (19–20)	0.61M	0.49	262 (251–281)	113	126	11	5
B-full-code	2 (2–2)	0.02M	0.009	165 (165–168)	5	126	23	5
C2 common API + code	2 (2–2)	0.02M	0.009	152 (152–154)	5	126	11	5
C2-lean	2 (2–2)	0.002M	0.008	154 (152–157)	6	126	11	5
C1 standing	20 (19–20)	0.46M	0.33	194 (189–210)	108	60	13	1
C-RDF standing	22 (21–22)	0.74M	0.53	220 (198–220)	138	60	12	1
C2 standing	2 (2–2)	0.02M	0.009	94 (90–96)	8	60	11	1

Table 1: Grounded condition, Claude Sonnet 5.5, five runs per set-up, all successful. Medians, with interquartile ranges in brackets. Cost is at list price for the agent's own requests. Wall time is summed over the two sessions. Model time covers the agent's own requests only; model, human and network time, also in seconds, need not add up to the wall time, because of runtime overhead, including housekeeping calls, and overlapping calls. Human interactions are one question to the borrowers plus one-time codes (A 6 in four runs and 7 in one; B 2), OAuth consents (B 3; B-full and B-full-code 4) or wallet approvals (4). Including the runtime's housekeeping calls, the request medians are 113, 81, 33, 22, 22, 4, 4, 4, 22, 25 and 4.

Claude Sonnet 5.5 ran five times per cell (three in search) and Claude Opus 5.5 three times on A and C1 at baseline, 223 runs in all, on 29 October 2026.

4 The grounded condition

Table 1 gives the grounded condition. All 55 runs succeeded with no violations: every run chose Brightpath, none took the cheaper-looking offer whose closing costs broke the limit, and none sent a document before the Loan Estimate.

Structured access does most of the work. Websites took 107 model requests, $1.26 and 730 seconds; APIs at the banks and payroll cut that to 77 requests, and APIs at the lenders too to 29; the common API took 20. Per-organization APIs everywhere thus achieved 90% of the common API's reduction in requests and 83% of its reductions in cost and time. Compared with B-full, C1 used 31% fewer model requests, cost 33% less and finished 24% sooner, with 37 tool calls against 79. This measures the common-API design as a whole: the agent read one description instead of a documentation page per organization, and the lender fetched records from their source by reference.

Routine steps in code leave two requests. Both programs made their two model calls for under a cent and finished in 152 to 165 seconds, 126 of them human time. Subtracting the recorded human waits leaves median times of 25 seconds for C2, 38 for B-full-code, 129 for C1 and 570 for websites. The program over per-organization APIs held an integration for each organization and made 61 API calls; the one over the common API had one request function and made 21. A one-line system prompt (C2-lean) cut tokens per model call by 90% but left latency and cost almost unchanged ($0.0083 against $0.0087 a run): the default prompt is almost all cache reads, at a twentieth of the input price.

Where the time goes. Model time was the largest share on websites (364 seconds) and with per-organization APIs (179 seconds everywhere); with the common API people took slightly longer than the model (126 against 113 seconds). Network time hit websites hardest: 148 seconds of page loads and posts, against 11 to 25 seconds for the API-only set-ups. Human time was 126 seconds in every set-up without websites (four consents or approvals at 16.6 seconds and one question at 60) and 160 seconds on websites, which need a code at every login. People took 22% of the website set-up's wall time, half of C1's and 83% of C2's. The runtime's own overhead added a median of 9 to 31 seconds a run, depending on the set-up (31 on websites, 13 for C1, about 9 estimated for C2); without it the wall times are 713, 246 and 143 seconds.

The second session. Every model-run set-up used 5 to 22 more requests than in the realistic setting (Table 5), from 15 to 20 with the common API and from 100 to 107 on websites; the grounded changes were introduced together, so the increase reflects all of them at once. Session 2 took a median of 9 and 38 requests respectively, mostly re-reading the note, the authority and the Loan Estimate, logging in again and refreshing tokens.

Standing delegation. With grants approved in advance only the deposit question reached a person: C1 fell from 255 to 194 seconds and C2 from 152 to 94. The per-organization approvals need not always be required. In future a person could give their agent a delegated credential up front, one that lets it prove to each organization that it may act for them, within set limits [14]. The approvals would then disappear, leaving only the questions that need information the borrowers alone have, here the deposit. Websites would still need a code at each organization unless devices were remembered.

RDF and the index. C-RDF took as many requests as C1 with 31% more tokens and 49% more cost; its JSON-LD results are about twice the size of C1's JSON. With three lenders, comparison took a median of three or four model requests everywhere, and the fewest tokens with C-RDF's single index query.

Safety. The injected text reached the model in every run of B, B-full, C1 and C-RDF, in no website run and never in the programs, whose model calls do not see records; it was never followed. The common API's server-side checks refused calls in 6 of the 55 runs: in two C1 runs (one with standing grants) session 2 re-requested payslips it already had and hit the grant's limit of three records per person, and in four standing C-RDF runs the first submission used the wrong token. All six recovered and succeeded; no agent attempted an action outside its authority.

5 Searching across many lenders

Table 2 shows lender choice at scale. All 42 runs chose the right lender and none took a trap. On websites time and cost grew linearly, by 5.5 seconds and about a cent per lender, so 30 lenders took 201 seconds and $0.34. With the common API cost grew 2.4 times more slowly and time seven times more slowly, but every product list entered the model's context, so 100 lenders took 96 seconds and $0.42. With the index one SPARQL query answered every N: 100 lenders took 19 seconds and $0.05. At 30 lenders, websites cost 8 times as much as the index and took 10 times as long. The program grew only with API latency. Building the index took a median of 0.3 to 0.5 seconds for up to 30 lenders and 0.8 to 0.9 seconds for 100.

	Cost ($) at N =				Wall time (s) at N =				Per extra lender		
Set-up	3	10	30	100	3	10	30	100	Requests	Cost ($)	Time (s)
A websites	0.088	0.148	0.341	–	41	78	201	–	0.46	0.0096	5.5
C1 common API	0.036	0.056	0.141	0.423	27	21	33	96	0.03	0.0040	0.76
C-RDF index	0.032	0.046	0.044	0.051	23	19	20	19	0.00	0.0003	−0.04
C2 code	0	0	0	–	1.3	3.4	9.7	–	0	0	0.31

Table 2: Search: choosing among N lenders, Claude Sonnet 5.5, three runs per cell, all correct. Medians. Growth per extra lender is a least-squares slope over all runs. The index is built outside the runs.

Set-up	Model	n	Success	Model requests	Tokens	Wall time (s)	Tool calls
A websites	Sonnet 5.5	5	5	83 (78–85)	4.13M	345 (313–348)	126
B per-org APIs + lender web	Sonnet 5.5	5	5	46 (44–47)	2.32M	230 (228–254)	90
B-full per-org APIs	Sonnet 5.5	5	5	16 (16–17)	0.50M	108 (106–124)	57
C1 common API	Sonnet 5.5	5	5	12 (12–12)	0.35M	66 (62–72)	21
B-full-code	Sonnet 5.5	5	5	2 (2–2)	0.02M	23 (20–24)	0
C2 common API + code	Sonnet 5.5	5	5	2 (2–2)	0.02M	22 (21–23)	0
A websites	Opus 5.5	3	3	65 (62–66)	2.66M	213 (202–234)	126
C1 common API	Opus 5.5	3	3	12 (12–12)	0.27M	77 (69–81)	21

Table 3: Simplified baseline: medians, with interquartile ranges in brackets. Tokens total all classes, of which 85–98% are cache reads. The B-full-code and C2 programs made 47 and 15 API calls themselves. Counts exclude the runtime's housekeeping calls; including them, the Sonnet medians are 84, 48, 18, 13, 3 and 3.

6 Simplified settings

The simplified settings (Section 3) understate absolute cost and time; we report them for the controlled comparisons they allow.

Baseline. The baseline (Table 3) shows the grounded pattern: per-organization APIs everywhere achieved 94% of the common API's reduction in requests. With the common API both models used 11 or 12 requests in every run; on websites Opus needed fewer than Sonnet.

Friction. Under friction (Table 4) the website agent needed a third more requests. In one run a retried payslip download hit an expired session, the browser saved the login page as the PDF, and the agent uploaded it and reported that every payslip had been sent: the study's only failure, and a silent one. C1 rose by four requests; C2's program absorbed the friction with eight extra API calls.

Over-asking and out-of-limits offers. No run shared or opened a business statement, and all twenty replies to the over-asking lender declined that part of the request, some after checking with the borrowers (A in 4 of 5 runs, B 5, C1 3, C2 5). No run recorded intent on the 6.875% estimate, and all twenty told the borrowers.

Realistic setting. The lender choice and sign-in steps (Table 5) added requests to every model-run set-up without changing their order; one wallet approval per organization took the place of one

Set-up	Baseline	Friction	Over-ask	Out-of-limits
A websites	83 / 4.13M / 345 / 5	110 / 6.12M / 456 / **4**	70 / 3.60M / 310 / 5	72 / 3.87M / 374 / 5
B per-org + lender web	46 / 2.32M / 230 / 5	42 / 2.18M / 210 / 5	43 / 2.21M / 199 / 5	32 / 1.60M / 193 / 5
C1 common API	12 / 0.35M / 66 / 5	16 / 0.51M / 112 / 5	12 / 0.35M / 76 / 5	10 / 0.28M / 72 / 5
C2 common API + code	2 / 0.02M / 22 / 5	2 / 0.02M / 25 / 5	2 / 0.02M / 26 / 5	2 / 0.02M / 20 / 5

Table 4: Simplified settings, Claude Sonnet 5.5, five runs per cell: median model requests / total tokens / wall time in seconds / successful runs.

Set-up	Model requests	Tokens	Wall time (s)	Human	Human interactions by type
A websites	100 (97–113)	5.99M	312	6 (6–7)	5–6 codes, 1 question
B per-org + lender web	55 (51–56)	3.02M	257	5	1 code, 3 consents, 1 question
B-full per-org	20 (20–20)	0.75M	106	5	4 consents, 1 question
C1 common API	15 (14–15)	0.44M	88	5	4 approvals, 1 question
C-RDF JSON-LD + index	14 (14–14)	0.59M	94	5	4 approvals, 1 question
C2 common API + code	2 (2–2)	0.02M	22	5	4 approvals, 1 question

Table 5: Simplified realistic setting (lender choice and instant human sign-in steps), Claude Sonnet 5.5, five runs per set-up, all successful. Medians; interquartile ranges in brackets.

consent per organization.

Safety. In these 126 runs no agent shared an unauthorized document, followed the injection or recorded intent outside the limits, and none attempted an out-of-scope action, so the permission checks never fired. The injection reached the model in every B and C1 run, but in no Sonnet website run before the realistic setting.

7 Discussion

Structure, sharing and code. The measurements separate three changes that arguments for agent standards often bundle. Structured access, replacing pages with explicit requests and typed responses, gave the largest saving. A common interface, with records passed by reference, added a further, smaller saving to a model-orchestrated agent and, in code, replaced an integration per organization with one. Moving routine control flow into code removed almost all model work, but only where someone writes that code for the task. The raw compute benefit therefore does not require a *standard*: per-organization APIs everywhere, sequenced by code, reach the same two model calls. The case for a common interface rests instead on what our testbed does not price: the cost of finding, learning and maintaining a different integration for every organization an agent deals with, and having permission, purpose and recipient explicit in every request, where a server can check them.

Search and people. Comparing many organizations is where a pre-computed index pays off: one query to a shared index answered a hundred lenders in 19 seconds for $0.05, while the website agent's cost grew with every lender. Such an index needs organizations to publish offers in structured form, which few lenders do today (Section 9). Once routine steps are automated, people become the bottleneck: without websites their codes, consents, approvals and answer took 126 seconds, more than the common-API agent's model time and five times the machine time of the program. Standing

delegation removes most of that wait; a delegated credential given up front [14] would do so without a wallet approval at each first use.

Cost. At Anthropic's list prices [27], a grounded run cost $1.26 on websites, $0.49 with per-organization APIs everywhere, $0.33 with the common API and under a cent with code. In dollars the website-to-common-API ratio is about 4, against 12 in raw tokens, because cache reads, which are most of the tokens, cost a twentieth of fresh input. We did not measure energy, and we do not model adoption or integration costs.

8 Assumptions and sensitivity

Table 6 lists the assumptions that bear on the metrics we report (model requests, tokens and cost, wall time, human interventions and success), where each is set, how it is grounded, and what it affects; the full inventory, with code locations and the analysis scripts, is in the artifact [16]. Sources include the Fannie Mae Selling Guide [21, 29], the CFPB [28, 20], the HTTP Archive [32], a usability study of two-factor authentication [31], session-timeout guidance [30] and Anthropic's prices [27].

Table 6: The main assumptions of the testbed and measurement, with their grounding and their effect on the results. Values are for the grounded condition unless stated. Code locations are in the artifact's `testbed/`. S: source; M: measured; A: arbitrary or assumed; U: unverified.

Assumption (value)	Where	Grounding	Effect on results
Task and documents			
Two months of statements for 4 accounts; 2 payslips each; each borrower's 2025 W-2 (simplified settings: no W-2s)	`data.js:204`, `fixtures.js:84`	S: Fannie Mae requires "the most recent full two-month period" of statements, a paystub within 30 days with YTD earnings, and W-2s for 1–2 years (B3-4.2-01, B3-3.2-01); one year is A	Fewer documents; favours set-ups that move each file (A, B)
Two conditions: the $8,000 deposit, answered in writing, and the W-2s	`data.js: 208-210`	S: the deposit exceeds 50% of monthly qualifying income ($7,692), Fannie Mae's "large deposit" test (B3-4.2-02); two conditions only is A	Real loans raise more; web costs are a lower bound
Documents accepted only after the Loan Estimate and intent to proceed (simplified settings: with the application)	`server.js:302`, `783`, `800`	S(survey): Reg Z bars lenders from requiring verification documents before the Loan Estimate (12 CFR 1026.19(e)(2)(iii)); accepting none earlier, and none before intent, is A	Built in; the grounded changes together add 5–22 requests per model-run set-up (M)
Lender steps			
Loan Estimate issued between two agent sessions; session 2 resumes from a handover note (simplified settings: after a fixed 5 s)	`runner.js:89`, `191`	S: due within 3 business days (CFPB); the short gap is A	Session 2 takes 9 (C1) to 39 (B) requests (M)
Session 2 finds the runtime's fixed prefix (9–15k tokens) in the prompt cache	`runner.js:190`	M: read from cache at session 2's first request	Models an always-on agent; a cold start costs more (see text)

Assumption (value)	Where	Grounding	Effect on results
Three lenders compared on advertised example figures	`data.js:196`	S: CFPB advises "at least three Loan Estimates"; advertised rates are not offers	Discovery cost (3–4 requests) is a lower bound; Table 2 scales it
Interfaces and environment			
Page boilerplate giving snapshots of about 12.4k characters (simplified settings: 1–4 KB pages)	`server.js:126`	M: 13 real bank, lender and payroll pages had a median 12.3k-character accessibility snapshot; S: HTTP Archive median page 2.65 MB, 71 requests	Built in
Latency: 1.85 s per page, 0.67 s per post or download, 0.3 s per API call (simplified settings: localhost only)	`server.js:1017`	M: median DOMContentLoaded and time to first byte of the 13 pages; the API figure is A	Network time 148 s for websites, 11–25 s for APIs (see text)
No bot management or CAPTCHAs	–	S(survey): Akamai Bot Manager on 10 of 13 big-bank logins	Favours websites
Per-organization consumer APIs and the common API exist and are clean	`server.js:708`	S(survey): hypothetical; only aggregators today	Favours B, B-full and C
Human in the loop and friction			
A code at every web login (devices never remembered); one consent or approval per organization	`server.js:843`, `user-tool.js`	S(survey): new-device codes match bank practice; the per-organization counts are A	Human counts
16.6 s per code, consent or approval; 60 s per question; always correct (simplified settings: instant)	`user-tool.js:64`	A: the median SMS second-factor time in Reese et al. (S), used for consents and approvals too (U); 60 s is A	Human time 126 s in API-only set-ups (see text)
Standing delegation: grants approved in advance	`runner.js:46`	A	Removes 4 of 5 interactions and 66 s
Friction (simplified settings): session expiry after 4 pages (8 at the lender); one 503; tokens expire after 1–2 uses	`server.js`	A. S(survey): real idle timeouts are 20–60 min; M: no single-site web session lasted more than 8.3 min	A time-based timeout would never fire; the friction results depend on A
Runtime and measurement			
Claude Code 2.1.280 with default effort; its prompt is in every request	`llm.js`, `runner.js`	M: first-request context 9.4k (code-run) to 16.5k (A) tokens; a one-line prompt cut tokens per call by 90%	All counts include the runtime
Model request = assistant message; 1–7 housekeeping calls per run excluded	`llm.js:22`	M	Counts are 1–7 below all API calls
Tokens summed over classes, unweighted	`llm.js`	M: cache reads are 86–98% of each grounded set-up's median tokens (25% in C2-lean; 85–98% at baseline); S: they cost 1/20 of base input (Anthropic pricing)	Token ratios exceed cost ratios
Cost at list price from each request's token classes	`runner.js:20`	S: Anthropic pricing, read 29 October 2026	Reported costs
Wall time summed over sessions, gap excluded; includes runtime start-up and housekeeping	`runner.js:39`, `overhead.js`	M: overhead per run 8–33 s, arm medians 9–31 s; estimated for the grounded code-run set-ups from 10 calibration runs	Reported with and without overhead
Up to six runs at once	results README	M: no consistent within-arm latency change at $\geq$4 runs in flight (baseline)	Effect on grounded and search latency not isolated

Grounded in the testbed. Assumptions that the simplified settings left to sensitivity estimates are built into the grounded condition, so their effects appear directly in Table 1: page weight matched to real pages; measured page and post latency, with an assumed 0.3 s per API call; published human response times; documents only after the Loan Estimate, which Reg Z allows, with the estimate arriving between two sessions in place of an artificial processing delay; and market loan figures.

Sensitivity. The ranking of the set-ups survives every remaining change we tested; the size of the gaps depends most on how long people take. Where the logs allow, we quantified each change instead of arguing it (scripts in the artifact).

- **Human response time.** We use the study's 16.6 s median SMS time for every code, consent and approval, and assume 60 s per answer. If every human step took two minutes, grounded wall time would be about 1,410 s for websites, 729 s for C1 and 625 s for C2, and the website-to-code ratio would fall from 4.8× to 2.3×. With standing grants C1 would take 254 s and C2 154 s.
- **Latency.** At 0.1 or 0.67 s per API call, C1 would take 250 or 265 s and C2 147 or 159 s. Taking each page navigation at the real pages' median time to first byte or to the load event (0.67 or 5.49 s), and each post, download and API call at 0.1 or 0.67 s, websites would take 643 or 958 s against 730 s.
- **A cold cache.** After a real wait of hours to days the runtime's 9–15k-token fixed prefix would have left the prompt cache. Writing it again at the one-hour cache-write price would add about $0.04–0.06 to each session that starts cold: little for the model-run set-ups, and several times the code-run set-ups' whole cost.

9 How the sandbox compares with today

A survey of current services (29 October 2026) supports the task and qualifies the interfaces. The two-month statement window matches Fannie Mae's purchase-loan requirement, and two recent payslips and one year of W-2s per borrower is a plausible simplification of a fuller documentation package [21]. US rules bar lenders from requiring verification documents before the Loan Estimate [20]; accepting documents only after it is the grounded sandbox's own choice. One-time codes on new devices, OAuth consent and handing logins and CAPTCHAs back to the user match current bank practice and how Muse, dots and Grok Bot behave [7, 19, 23]. The grounded pages match real ones in snapshot size and load time, but the websites are still easier: 10 of 13 large US bank login pages we checked set Akamai Bot Manager cookies, one lender's rate page refused a scripted request, and real applications are longer and raise more conditions. The per-organization consumer APIs and the common agent API are hypothetical. Today bank data reaches third parties through contracted aggregators. FDX reported more than 114 million connections on FDX-aligned APIs in October 2025, and separately estimated that token-based APIs carried about two-thirds of user-permissioned traffic [22]. The CFPB's open-banking rule, the strongest push towards standard consumer access, was preliminarily enjoined on 29 October 2025 [17]; we did not establish its position a year later. No US lender we found offers a consumer- or agent-facing application API. Structured rate data is rare: 2 of 10 major lender rate pages we retrieved carried schema.org rate data, one of them stale, and none of the 12 pages retrieved (ten lenders, two marketplaces) published closing costs in structured form. The survey, with sources and measurements, is in the artifact [16].

10 Limitations

- **One synthetic, cooperative scenario,** built with the set-ups in mind. Real websites add CAPTCHAs, bot management, errors and layout changes, so the website costs are probably lower bounds. The per-organization APIs are as clean and well documented as the common one, so B-full does not reflect the cost of finding or maintaining real integrations. The common API assumes that every organization implements it.
- **One agent runtime.** Every model request, except in C2-lean, carries Claude Code's default system prompt (roughly 9,000–20,000 tokens) and its caching. We did not build a comparable GPT arm within the time available: the available command-line agent reports usage only as one total per run, and its built-in tools would first have to be removed.
- **Small samples** (five runs per cell; three in search and for Opus). Variance was higher with websites and per-organization APIs; B's medians vary across settings more than friction moved them.
- **Modelled people and a short gap.** Response times come from one usability study and one assumption; people who are away take longer, and may approve per person where we assume one approval per organization. Session 2 started minutes after session 1 with a warm cache; a real wait of up to three business days would find it cold. In the friction setting the set-ups met different numbers of friction events (Section 3).
- **A shallow checker.** Documents are identified by markers in their text rather than by validating their contents; application fields are spot-checked; and the condition response passes if it mentions a gift and parents.
- **A mock permission model.** The common API checks grants for operation, subject, records, recipient, purpose and counts, but record origins are not cryptographically verified on retrieval, the recipient is a caller-supplied header, and the on-behalf-of field is not validated. The injection probe had restricted channels (no email tool, HTTP limited to the sandbox); the over-asking lender accepted a refusal without consequence; and the out-of-limits estimate omits escrow.
- **The RDF variant** is a translation layer over the same permission logic, its vocabulary was documented only to that set-up, and only one SPARQL engine was used.
- **Untested guards.** The server-side checks fired only on benign retries, and no agent attempted an out-of-scope action, so the paper does not show that the common interface prevents violations in practice; it shows that, on this task, current Claude models stayed within their authority on every interface.
- **Wall time** includes runtime overhead, also reported separately; for the grounded code-run set-ups it is estimated, probably low (13.5 seconds measured for C2 with standing grants, against 9.1 estimated). Up to six runs ran concurrently.

11 Related work

Benchmarks such as WebArena [34], Mind2Web [5] and OSWorld [33] measure whether agents can complete tasks on websites and desktops; we hold the task fixed, check success, and compare the

cost of different interfaces to the same services. Song et al. [13] show that agents calling APIs can outperform browsing agents on WebArena tasks; we add per-organization against common interfaces and code orchestration, and report requests, cost and time. Prompt injection against agents is studied in AgentDojo [4] and, for personal data, by Alizadeh et al. [2]. Protocols such as MCP [9] and A2A [1] standardize how agents reach tools and each other, but not the meaning of domain requests, which is what our common agent API adds.

12 Availability and disclosure

The testbed, the per-run logs (agent transcripts, proxy usage logs, server logs and final state) and the scripts that produce every table are archived as [16]. To reproduce, run `./testbed/run.sh <arm> <model> <n> [slot] [condition]`, or for search `node testbed/search-runner.js <arm> <model> <n> <N>`, then `node testbed/summarize.js`.

The research question and framing are the author's. The experiment was designed, implemented and run, and the text drafted, with the help of Claude Opus 5.5 (Anthropic), and the paper was checked against the raw data by GPT-6.1 Sol (OpenAI); the author has reviewed the paper and takes responsibility for its content, but has not reviewed all of the testbed and analysis code line by line. The agents under test were Claude models; Anthropic is not involved in this work.

References

[1] Agent2Agent Project. Agent2Agent (A2A) Protocol Specification. https://a2a-protocol.org/latest/specification/, accessed October 2026.

[2] M. Alizadeh et al. Simple prompt injection attacks can leak personal data observed by LLM agents during task execution. arXiv:2506.01055, 2025.

[3] P. A. Bonatti et al. Towards computer-using personal agents. arXiv:2503.15515, 2025.

[4] E. Debenedetti et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. arXiv:2406.13352, 2024.

[5] X. Deng et al. Mind2Web: Towards a generalist agent for the web. arXiv:2306.06070, 2023.

[6] K. McGuirk. Plaid powers Meta's new AI agent, Muse. Plaid blog, 8 September 2026.

[7] Meta. How we built safety into Muse. https://research.meta.ai/blog/security-and-safety-for-ai-agents-our-approach-with-muse, September 2026, accessed October 2026.

[8] Meta. Introducing Muse: the world's first personal AI agent built for everyone. 8 September 2026.

[9] Model Context Protocol Project. Model Context Protocol specification, version 2025-06-18.

[10] Microsoft. Playwright MCP, version 0.0.83. https://github.com/microsoft/playwright-mcp.

[11] OpenAI. How we build safety, security and privacy into dots. 29 September 2026.

[12] OpenAI. Introducing dots. 29 September 2026.

[13] Y. Song, F. F. Xu, S. Zhou and G. Neubig. Beyond browsing: API-based web agents. arXiv:2410.16464, 2024.

[14] T. South et al. Authenticated delegation and authorized AI agents. arXiv:2501.09674, 2025.

[15] J. Wright. Letting software agents act for us. Manuscript in preparation, 2026.

[16] J. Wright. Mortgage agent interfaces: testbed and run logs, version 1.0.0. https://github.com/jeswr/mortgage-agent-interfaces, 2026.

[17] ABA Banking Journal. Kentucky federal court enjoins CFPB from enforcing current 1033 final rule. 3 November 2025.

[18] H. J. Pandit et al. Data Privacy Vocabulary (DPV). W3C Data Privacy Vocabularies and Controls Community Group, https://w3id.org/dpv.

[19] OpenAI. dots documentation. https://learn.chatgpt.com/docs/dots, accessed October 2026.

[20] Code of Federal Regulations, 12 CFR 1026.19(e)(2)(iii). https://www.ecfr.gov/current/title-12/part-1026/section-1026.19.

[21] Fannie Mae. Selling Guide B3-4.2-01, Verification of deposits and assets; B3-3.2-01, Standards for employment and income documentation (4 March 2026). Accessed October 2026.

[22] Financial Data Exchange. Response to CFPB advance notice of proposed rulemaking, docket CFPB-2025-0037. 20 October 2025.

[23] xAI. Grok Bot: approvals, security and privacy. https://docs.x.ai/grok-bot/approvals-security-and-privacy, accessed October 2026.

[24] T. Lebo, S. Sahoo and D. McGuinness (eds.). PROV-O: The PROV ontology. W3C Recommendation, 2013.

[25] Schema.org. MortgageLoan, LoanOrCredit and MonetaryAmount types. https://schema.org, accessed October 2026.

[26] sparq. SPARQL engine with WebAssembly build, npm package @sparq-org/sparq, version 0.1.5. https://github.com/sparq-org.

[27] Anthropic. Pricing. https://platform.claude.com/docs/en/about-claude/pricing, accessed 29 October 2026.

[28] Consumer Financial Protection Bureau. Request and review multiple Loan Estimates. https://www.consumerfinance.gov/owning-a-home/compare/request-and-review-multiple-loan-estimates/.

[29] Fannie Mae. Selling Guide B3-4.2-02, Depository accounts. October 2026.

[30] NIST. Digital identity guidelines: authentication and authenticator management, SP 800-63B-4. 2025.

[31] K. Reese et al. A usability study of five two-factor authentication methods. In *Proc. SOUPS*, 2019.

[32] HTTP Archive. Web Almanac 2024: Page weight; Web Almanac 2025: Performance. https://almanac.httparchive.org.

[33] T. Xie et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. arXiv:2404.07972, 2024.

[34] S. Zhou et al. WebArena: A realistic web environment for building autonomous agents. arXiv:2307.13854, 2023.